

day21笔记

一、RAG介绍

1、什么是RAG？是检索增强生成

2、为什么需要RAG？

- LLM的训练数据是有截止日期的，无法回答最新问题的

示例代码

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
load_dotenv()

# llm = ChatTongyi(model='qwen3-max')
# result = llm.invoke('你最新的训练数据是哪一年多一天的? ').content
# print(result)

llm = init_chat_model(model='deepseek-chat')
result = llm.invoke('你最新的训练数据是哪一年多一天的? ').content
print(result)
```

输出结果

我的知识截止日期是2025年5月。但具体到“哪一年多一天”这样的细节，由于训练数据涵盖多种来源和不同时间点的信息，无法精确到“某一年比标准年份多一天”这种特殊日期。我的回答基于截至2025年5月的公开知识，如果需要最新动态，建议通过联网搜索功能查询实时信息。

- 幻觉问题，生成一些答案看似合理其实是错误的（因为AI它的循环语料是日期的，你问的问题超出了它的限制，这个时候又不好不给回答，因此会根据以往的经验给你一个答案）
- 缺乏专业知识，LLM可能不了解你的公司、品牌、内部的知识

以上存在的问题，可以通过RAG来解决

3、RAG解决方案

- 用户提问
- 检索相关知识（搭建本地知识库）
- 将检索出来的知识注入给提示词
- 生成精准的答案

二、RAG文档加载

1、安装

```
# pypdf 加载pdf文档
# unstructured 多种文档格式支持
# selenium 处理网络加载
pip install pypdf unstructured selenium

# 多文档格式支持“全家桶”
pip install "unstructured[all-docs]"
```

2、文本文件加载

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_community.document_loaders import TextLoader
from dotenv import load_dotenv
load_dotenv()

loader = TextLoader('./file/用户问题.txt', encoding='utf-8')

documents = loader.load()

# 注意点：被加载出来的数据放在一个列表中的、
# page_content 存放文档内容的属性
# metadata 元数据（表示这个数据的来源是什么）
# print(documents[0].page_content)
# print(documents[0].metadata)

for doc in documents:
    print(doc.page_content)
    print('-' * 30)
    print(doc.metadata)
```

3、CSV文件加载

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_community.document_loaders import CSVLoader
from dotenv import load_dotenv
load_dotenv()

loaders = CSVLoader(
    './file/员工数据表.csv',
    encoding='utf-8',
    source_column='姓名' # 指定以什么作为依据（某个数据来源于哪里）
)

documents = loaders.load()

for doc in documents:
    print(doc.page_content)
    print('-' * 30)
    print(doc.metadata)
```

4、PDF文件加载

```

from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_community.document_loaders import PyPDFLoader
from dotenv import load_dotenv
load_dotenv()

loaders = PyPDFLoader('./file/提示词工程.pdf')

documents = loaders.load()

# Ignoring wrong pointing object 45 0 (offset 0) 大家以后看到这个提示，其实不是加载错误，而是表示加载的pdf文档有损坏，或者内容格式存在某些问题
# print(documents)

print(f'共{len(documents)}页')

# 通过索引查看那些页内容
target_pages = documents[0:3]

print(f'查看，共{len(target_pages)}页')

for i, doc in enumerate(target_pages):
    print(f'--- 第{i+1}页 ---')
    print(doc.page_content)

```

5、批量加载文件

```

from langchain_community.document_loaders import DirectoryLoader, TextLoader
from dotenv import load_dotenv
load_dotenv()

# 加载某个文件夹下的所有文件
# *.txt 表示匹配当前目录下所有的.txt文件 例如: a.txt b.txt hello.txt
# **/ 表示递归匹配，意思是不管在那个文件夹下，都可以匹配到
# **/*.txt 整体的意思是可以匹配到./file文件夹下以及所以子文件夹里面的文件。文件后缀做了限定，只能匹配到.txt文件

# loader_cls=TextLoader 使用那个模块加载文件
# show_progress=True 表示显示加载文件的进度
# loader_kwargs={'encoding': 'utf-8'} 设置字符编码
loader = DirectoryLoader(
    path='./file',
    glob='**/*.txt',
    loader_cls=TextLoader,
    show_progress=True,
    loader_kwargs={'encoding': 'utf-8'}
)

documents = loader.load()
print(f'共加载了{len(documents)}个文件')

for document in documents:
    print(document.page_content)

```

```
print('-' * 30)
print(document.metadata)
```

6、Markdown文件加载

安装

```
pip install markdown
```

示例代码

```
from langchain_community.document_loaders import UnstructuredMarkdownLoader
from dotenv import load_dotenv
load_dotenv()

loaders = UnstructuredMarkdownLoader('./file/day19笔记.md')

documents = loaders.load()

print(documents[0].page_content)
```

7、网页加载

```
import os
# 给langchain查看的，告知身份，一定要放在最前面
os.environ["USER_AGENT"] = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'

from langchain_community.document_loaders import WebBaseLoader

headers = {'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36'}
# headers, 给WebBaseLoader查看的，如果不添加也会出现这个信息
# loader = WebBaseLoader(
#     'https://www.baidu.com',
#     requests_kwargs={'headers': headers},
# )

# 加载多个网页
loader = WebBaseLoader(
    ['https://www.baidu.com', 'https://www.jd.com'],
    requests_kwargs={'headers': headers},
)

documents = loader.load()

for doc in documents:
    print(doc.page_content.strip())

print('-' * 50)
```

三、TextSplitter 文档分隔

1、文档为什么要进行分隔？

- 问题：文档太长了，无法一次全部放进提示词
- 解决方案：进行分隔，可以把大文档切割成小块。例如：原始文档 10000字，切成4块，每块，就是2500字

2、分隔方法

- `split_document`，输入和输出类型都是一个列表。推荐，当咱们加载完文档后，直接使用这个方法，能保留文件名和元数据
- `split_text`，输入类型是字符串，输出类型是一个列表，当咱们加载完文档后，只是字符，没有元数据

3、示例代码

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# RecursiveCharacterTextSplitter 递归式字符文本分割器（推荐方式）

# text = """
# 人工智能（Artificial Intelligence, AI）是计算机科学的一个分支，
# 旨在开发能够执行通常需要人类智能的任务的系统。
#
# 机器学习是人工智能的一个子领域，
# 专注于开发能够从数据中学习的算法。
#
# 深度学习是机器学习的一个分支，
# 使用神经网络进行特征学习。
# """
# chunk_size 文档分隔每块的大小（字符数）
# overlap 指定是块与块之间重叠的部分，为了保证语义的完整性
# length_function 表示计算字符长度的函数
# separators 表示按照什么符号进行切割，依次进行降级处理（根据不同的情况）
"""
前一个 chunk 的结尾是“该函数的主要作用是”，而下一个 chunk 的开头是“处理用户登录请求”。
如果没有重叠，检索时可能只拿到其中一半，导致大语言模型（LLM）无法理解完整语义。重叠部分确保了即使信息在切割点
附近，
也能被完整地包含在至少一个 chunk 中。
"""

# splitter = RecursiveCharacterTextSplitter(
#     chunk_size=200,
#     chunk_overlap=10,
#     length_function=len,
#     separators=["\n\n", "\n", "。", " ", " "],
# )
#
# chunks = splitter.split_text(text)

# print(chunks)

# for chunk in chunks:
#     print(chunk)
```

```

text = "第一段文本\n第二段文本\n第三段文本"
splitter = RecursiveCharacterTextSplitter(
    chunk_size=100,
    chunk_overlap=20,
    separators='\n'
)

chunks = splitter.split_text(text)

print(chunks)

for chunk in chunks:
    print(chunk)

```

四、Markdown分割器

```

from langchain_text_splitters import MarkdownTextSplitter

markdown_text = """
# 第一章 机器学习

## 1.1 什么是机器学习

机器学习是人工智能的一个重要分支...

## 1.2 机器学习类型

### 监督学习
监督学习是最常见的机器学习类型...

### 无监督学习
无监督学习不需要标签数据...
"""

splitter = MarkdownTextSplitter(
    chunk_size=100,
    chunk_overlap=20
)

chunks = splitter.split_text(markdown_text)

for chunk in chunks:
    print(chunk)

```

五、添加元数据

1、读取出来的文档会自带元数据，为什么还需要自己添加呢？

- 通过API接口请求回来的数据
- 自定义数据
- ...

2、示例代码

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document

text = "第一段文本内容。第二段文本内容。第三段文本内容。"

splitter = RecursiveCharacterTextSplitter(chunk_size=20, chunk_overlap=2)

chunks = splitter.split_text(text)

# 创建带元数据的文档
documents = [
    Document(
        page_content=chunk,
        metadata={'source': '文档A', 'chunk_id': i},
    )
    for i, chunk in enumerate(chunks)
]

for doc in documents:
    print(doc.page_content)
    print('-' * 30)
    print(doc.metadata)
```

六、向量存储与检索

1、向量数据库

向量数据库能把文本（我喜欢素食）转成一组数字（向量），然后存储起来。当你输入一个问题时，它也会转成向量，然后找出最相似的存储记录

2、工作原理

- 文本 --- 嵌入模型 --- 向量（一串数字）
- 向量存入数据库，同时保存原始文本
- 查询时，将查询文本转成向量，在数据库中距离最近的向量（计算，查看输入的问题和数据库里面的内容那个语义是最相近的）
- 找到对应向量，然后把原始文本返回

3、向量数据库安装

- ChromaDB 这个向量数据库是目前AI社区使用最多的一个，最大的特点就是对于初学者极为友好，主打一个轻量级和开箱即用
- 安装

```
pip install chromadb
```

七、向量数据库使用

```
from langchain_community.chat_models.tongyi import ChatTongyi
# RecursiveCharacterTextSplitter 分割器
from langchain_text_splitters import RecursiveCharacterTextSplitter
# DashScopeEmbeddings 阿里的文本嵌入模型（把文本进行向量化）
from langchain_community.embeddings import DashScopeEmbeddings
# Chroma 向量数据库
from langchain_community.vectorstores import Chroma
from dotenv import load_dotenv
from torch.ao.quantization.backend_config.onednn import embedding_op_dtype_configs
from torch.nn.utils import vector_to_parameters

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 准备数据 --- 由于不是读取的数据，因此需要把文本到时转成标准文档格式进行处理
texts = [
    "人工智能是研究如何让机器具有智能的学科",
    "机器学习是人工智能的一个分支，让计算机从数据中学习",
    "深度学习使用神经网络进行特征学习和预测",
    "Python是一种广泛使用的高级编程语言",
    "LangChain是一个用于构建LLM应用的框架"
]

# 创建文档分割器
# 行业标准做法: chunk_size=512, chunk_overlap=128 可以处理大部分的文档
splitter = RecursiveCharacterTextSplitter(
    chunk_size=60,
    chunk_overlap=10,
)

# create_documents(texts) 可以把文本转成标准文档格式再进行分隔
chunks = splitter.create_documents(texts)

# 创建嵌入模型
embeddings = DashScopeEmbeddings(model='text-embedding-v2')

# 创建向量数据库
# embedding 表示进行向量化处理
# documents 那个文档需要进行向量化
vectorstores = Chroma.from_documents(
    embedding=embeddings,
    documents=chunks,
    persist_directory='./chroma_db'
)

# 查看向量数据库里面存入了多少个向量
# print(vectorstores._collection.count())

# 进行相似度检索
```



```
query = '什么是机器学习?'

# k=2 表示最终返回两个语义相似的结果
result = vectorstores.similarity_search(query, k=2)

for i, doc in enumerate(result):
    print(f'{i}, {doc.page_content}')
```

八、RAG综合实战

```
from langchain_community.chat_models.tongyi import ChatTongyi
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.embeddings import DashScopeEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_community.document_loaders import TextLoader
from langchain.agents import create_agent
from langchain_core.tools import tool
from langchain_community.vectorstores import Chroma
from dotenv import load_dotenv

load_dotenv()

llm = ChatTongyi(model='qwen3-max')

# 1、加载文档
loaders = TextLoader('./file/颐和园.txt', encoding='utf-8')
documents = loaders.load()

# 2、文档分隔
splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=128,
)
chunks = splitter.split_documents(documents)

# 3、创建嵌入模型
embeddings = DashScopeEmbeddings(model='text-embedding-v2')

# 4、创建向量数据库
vectorstores = Chroma.from_documents(
    embedding=embeddings,
    documents=chunks,
    persist_directory='./chroma_db2'
)

# 5、使用搜索器
retriever = vectorstores.as_retriever(search_kwargs={'k': 2})

# 6、创建工具
@tool
def yiheyuan_search(query: str):
```

```
""" 回答关于颐和园相关的历史、景点、建筑或相关的问题，使用此工具进行查询（搜索） """
# 调用检索器获取文档列表
docs = retriever.invoke(query)

# 把检索到的内容返回出去
return '\n\n'.join([d.page_content for d in docs])

# 7、创建智能体
agent = create_agent(
    model=llm,
    tools=[yiheyuan_search],
    system_prompt='你是一个颐和园的智能导游助手。请根据用户的问题，利用提供的工具进行回答。如果工具中找不到答案，请根据你的通用知识回答，或者诚实地说你不知道。'
)

result = agent.invoke({'messages': [('user', '颐和园的长廊有多长? ')]})
print(result['messages'][-1].content)
```